

TESTE DE SOFTWARE

PROFESSOR: LEONARDO

www.leonardotirado.com





VAGAS DA SEMANA



ESTAGIÁRIO(A) DE QUALIDADE DE SOFTWARE (QA), TESTES E IA GENERATIVA



Presencial/Curitiba - PR

PESSOA ESTAGIARIA DE DESENVOLVIMENTO DE SOFTWARE



Híbrido/Curitiba - PR

AUXILIAR DE SUPORTE DE TI



Home Office/Curitiba - PR



EVENTOS DE TECNOLOGIA



TECHSTARS_STARTUP WEEKEND CURITIBA



SEBRAE/Curitiba - PR

CODECON SUMMIT



Expotrade/Curitiba - PR





FUNDAMENTOS DE TESTE DE SOFTWARE

DA TRAGÉDIA À RESPONSABILIDADE: CONSTRUINDO A BASE PARA PENSAR COMO QA

ROTEIRO DE HOJE



UNIANDRADE



✓ **Recapitulação: Therac-25**

Revisitando o caso e extraíndo lições para a aula de hoje

✓ **Verificação, Validação e Avaliação**

Os três pilares da garantia de qualidade de software

✓ **Defeito, Erro e Falha**

Precisão no vocabulário: a diferença entre o que o dev faz e o que o usuário vê

✓ **Os 7 Princípios do Teste (ISTQB)**

O DNA de qualquer estratégia de teste sólida

✓ **Pirâmide de Níveis de Teste**

Como organizar os testes ao longo do ciclo de desenvolvimento

✓ **Laboratório: Caça aos Bugs**

Atividade prática em duas trilhas — com ou sem programação



THERAC-25

O que nos ensina?

Na aula passada, vimos o caso do Therac-25, um erro de software que matou pacientes.
Mas qual foi a raiz do problema?



- > **Reutilização de código sem testes de regressão**
- > **Interface enganosa (mensagens de erro genéricas)**
- > **Confiança excessiva no hardware de segurança**
- > **Ausência de testes de integração**

THERAC-25

A lição que o mundo aprendeu



- > Testar não é "achar erro"! É garantir confiança
- > Um defeito escondido custa mais que um defeito encontrado cedo
- > O testador é o advogado do usuário

VERIFICAÇÃO × VALIDAÇÃO × AVALIAÇÃO

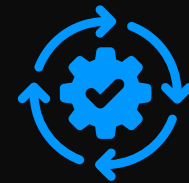
Três palavras que todo profissional de TI confunde. Vamos desenhar no quadro



Verificação (Verification)

"Estamos construindo o produto certo?"

- Revisões, walkthroughs, inspeções
- Checklists de requisitos
- Static analysis – (Análise estática)
- Não executa o código



Validação (Validation)

"Construímos o produto certo?"

- Testes funcionais e não-funcionais
- Protótipos com usuários
- Testes de aceitação (UAT)
- Executa o código



Avaliação

Processo sistemático de determinar se o produto atende aos critérios definidos. Engloba medição, análise e julgamento sobre a qualidade. É o "guarda-chuva" que abriga V&V.



DEFEITO, ERRO E FALHA

No dia a dia, todo mundo fala "tem um bug". Mas em Teste de Software, precisamos ser precisos:



Erro (Error / Mistake)

Ação humana incorreta. O dev escreveu $i \leq 10$ ao invés de $i < 10$.



Defeito (Defect / Bug)

A imperfeição no código. O $\leq$ permanece no software, é a semente do problema.



Falha (Failure)

O comportamento externo incorreto. O sistema cobrou R\$ 0,00 ou travou. É o que o usuário vê.



Regra de ouro: nem todo defeito causa falha. Um código morto pode ter defeitos que nunca se manifestam, mas são riscos dormindo.





OS 7 PRINCÍPIOS DO TESTE (ISTQB)

Esses princípios são o DNA de qualquer estratégia de teste sólida:



**TESTE REVELA
DEFEITOS**



**TESTES EXAUSTIVOS
SÃO IMPOSSÍVEIS**



**TESTE ANTECIPADO
ECONOMIZA**



**AGRUPAMENTO
DE DEFEITOS**



**PARADOXO DO
PESTICIDA**



**CONTEXTO
IMPORTA**



**A ILUSÃO DA
AUSÊNCIA DE
ERROS**



OS 7 PRINCÍPIOS DO TESTE (ISTQB)

1. Teste revela defeitos

Testar prova a presença, nunca a ausência de bugs.
Passar em todos os testes não garante software perfeito.



1. TESTE REVELA DEFEITOS

Exemplo:

Você testa um sistema de login e verifica:

- Login correto
- Senha incorreta
- Usuário inexistente
- Campos vazios

Isso não significa que o login está 100% sem bugs. Pode existir um problema, por exemplo, quando o usuário coloca uma senha com 500 caracteres ou utiliza determinados caracteres especiais.

encontrar 0 bugs não significa que não existam bugs.



OS 7 PRINCÍPIOS DO TESTE (ISTQB)

2. Testes exaustivos são impossíveis

Testar tudo é inviável. O segredo é priorizar riscos e usar técnicas sistemáticas.



2. TESTES EXAUSTIVOS SÃO IMPOSSÍVEIS

Exemplo:

Imagine um sistema de cadastro que possui:

- Nome, CPF, Data de nascimento, E-mail, Telefone e Senha.

Existem milhares ou milhões de combinações possíveis de dados.
Seria impossível testar:

"Vou testar todas as combinações de nome + CPF + data + e-mail + telefone + senha."

Em vez disso, você seleciona casos representativos e de maior risco.

Ideia principal: testar de maneira estratégica, e não tentar testar tudo.



OS 7 PRINCÍPIOS DO TESTE (ISTQB)

3. Teste antecipado economiza

Defeitos encontrados na análise de requisitos custam 100x menos que em produção.



TESTE ANTECIPADO ECONOMIZA

Exemplo:

Imagine que o programador recebe a seguinte regra:
"O usuário precisa ter pelo menos 18 anos."

Se essa regra for testada durante o desenvolvimento, o erro pode ser corrigido rapidamente.

Mas imagine que ninguém testa isso e o sistema é lançado. Depois um cliente descobre que uma pessoa de 15 anos consegue criar uma conta. Agora pode ser necessário:

- corrigir o código, atualizar o sistema, corrigir o banco de dados, investigar usuários afetados, fazer novo deploy e comunicar o problema.

Ideia principal: quanto mais tarde o defeito é descoberto, maior tende a ser o custo da correção.



OS 7 PRINCÍPIOS DO TESTE (ISTQB)

4. Agrupamento de defeitos

80% dos bugs costumam estar em 20% dos módulos.
Foque no código mais complexo.

```
self.file = None
self.fingerprints = set()
self.logdups = True
self.debug = debug
self.logger = logging.getLogger(__name__)
if path:
    self.file = open(os.path.join(path, 'fingerprint.log'), 'a')
    self.file.seek(0)
    self.fingerprints.update([fingerprint])

    @classmethod
    def from_settings(cls, settings):
        debug = settings.getboolean('DEBUG')
        return cls(job_dir(settings.get('JOB_DIR')), debug)

    def request_seen(self, request):
        fp = self.request_fingerprint(request)
        if fp in self.fingerprints:
            return True
        self.fingerprints.add(fp)
        if self.file:
            self.file.write(fp + '\n')

    def request_fingerprint(self, request):
        return request_fingerprint(request)
```



AGRUPAMENTO DE DEFEITOS

Exemplo:

Imagine um sistema com:

Login, Cadastro, Relatórios, Pagamento e Configurações

Depois dos testes, você descobre:

Login → 1 bug
Cadastro → 2 bugs
Relatórios → 15 bugs
Pagamento → 3 bugs
Configurações → 1 bug

Perceba que Relatórios concentra a maioria dos problemas.

Isso pode acontecer porque esse módulo:

- é mais complexo;
- possui mais código;
- foi desenvolvido às pressas;
- possui muitas regras;
- depende de outros sistemas.

Ideia principal: quando encontramos muitos bugs em uma determinada área, vale a pena concentrar mais testes nela.



OS 7 PRINCÍPIOS DO TESTE (ISTQB)

5. Paradoxo do pesticida

Repetir os mesmos testes não encontra bugs novos.
Varie técnicas e dados.



PARADOXO DO PESTICIDA

Exemplo:

Imagine que todos os dias você testa:

1. Abrir o sistema
2. Fazer login
3. Cadastrar usuário
4. Fazer logout

Durante vários dias tudo funciona.

Mas o sistema recebeu uma nova funcionalidade de exportação para PDF.

Se você continuar executando apenas os quatro testes antigos, provavelmente não encontrará problemas na nova funcionalidade.

Por isso, é necessário criar e atualizar os casos de teste.

É parecido com um pesticida: se você sempre utiliza o mesmo, os organismos podem se adaptar. Nos testes, repetir sempre os mesmos cenários pode fazer com que novos defeitos passem despercebidos.



OS 7 PRINCÍPIOS DO TESTE (ISTQB)

6. Contexto importa

Testar um app bancário é diferente de testar um blog. A estratégia muda.



CONTEXTO IMPORTA

Exemplo:

Compare:

Sistema bancário

Você precisa se preocupar muito com:

- segurança;
- valores financeiros;
- autenticação;
- permissões;
- integridade dos dados.

Sistema hospitalar

Pode haver uma preocupação ainda maior com:

- segurança;
- precisão das informações;
- disponibilidade;
- consequências de um erro.

Compare:

Jogo

Você provavelmente dará mais atenção para:

- desempenho;
- gráficos;
- controles;
- áudio;
- experiência do jogador;
- compatibilidade com diferentes dispositivos.

Ideia principal: o que é crítico em um sistema pode não ser crítico em outro.



OS 7 PRINCÍPIOS DO TESTE (ISTQB)

7. A ilusão da ausência de erros

Um software sem bugs que não atende às necessidades do usuário é inútil. Validação > Verificação pura.



A ILUSÃO DA AUSÊNCIA DE ERROS

Exemplo:

Imagine que você desenvolveu um aplicativo de transporte.

Você testa:

Login	OK
Cadastro	OK
Pagamento	OK
Solicitação corrida	OK
Cancelamento	OK

Nenhum bug foi encontrado.
Então você pensa:

"Meu aplicativo está perfeito!"

Mas os usuários dizem:

"É muito difícil encontrar o botão para chamar um carro."

Ou:

"O aplicativo demora muito para realizar uma tarefa."

Tecnicamente, talvez não exista um bug.
Mas o sistema não atende bem à necessidade do usuário.

Ausência de bugs $\neq$ software de qualidade.

PIRÂMIDE DE NÍVEIS DE TESTE

Como organizamos os testes ao longo do ciclo? Da base para o topo:



Testes de Aceitação (E2E)

Fluxo completo do usuário • Poucos testes • Alto custo • Alto valor de confiança

Testes de Integração

APIs, banco de dados, serviços • Médio volume • Isolam fronteiras

Testes Unitários

Funções e métodos isolados • Muitos testes • Rápidos • Baratos • Automatizados



Regra: quanto mais acima na pirâmide, mais lento e caro. Quanto mais abaixo, mais rápido e barato. Uma pirâmide invertida (muitos E2E, poucos unitários) é um sinal de dor.



ATIVIDADE



CAÇA AOS BUGS: SISTEMA DE RESERVA DE SALAS



Vocês receberam um cenário: um sistema de reserva de salas da universidade. O sistema permite que alunos reservem salas para estudo em grupo. Escolham uma das trilhas abaixo de acordo com seu nível de programação:



Trilha A — Teste Exploratório (sem código)

Você recebeu um documento de requisitos do sistema. Sua missão:

- Leia os 5 requisitos funcionais e identifique ambiguidades ou lacunas
- Simule 3 cenários de uso "estranhos" (ex: aluno tenta reservar sala para 50 pessoas)
- Para cada cenário, escreva: o que fiz → o que esperava → o que aconteceu → classificação (defeito / erro / falha)
- Prática: desenhe um fluxograma mostrando onde o sistema poderia quebrar



Trilha B — Análise de Código (com programação)

Você recebeu o código-fonte do módulo de reserva. Sua missão:

- Faça revisão estática do código (não execute ainda)
- Identifique pelo menos 5 defeitos potenciais (lógica, validação, segurança)
- Classifique cada um como: defeito, erro de design ou falha potencial
- Escreva 3 casos de teste que provocariam falhas reais no sistema
- Prática: proponha uma correção para o defeito mais grave



SAIBA MAIS:

[HTTPS://WWW.YOUTUBE.COM/WATCH?V=LJ-YS1J8TNS](https://www.youtube.com/watch?v=LJ-YS1J8TNS)

[HTTPS://WWW.YOUTUBE.COM/WATCH?V=8GKAAJCRXUW](https://www.youtube.com/watch?v=8GKAAJCRXUW)

[HTTPS://WWW.YOUTUBE.COM/WATCH?V=FCMYM01WFLC](https://www.youtube.com/watch?v=FCMYM01WFLC)

[HTTPS://WWW.YOUTUBE.COM/WATCH?V=UMMKGI1JBRQ](https://www.youtube.com/watch?v=UMMKGI1JBRQ)

[HTTPS://WWW.YOUTUBE.COM/WATCH?V=P-8SVWLVT2S](https://www.youtube.com/watch?v=P-8SVWLVT2S)



REFERÊNCIAS

PEZZÈ, Mauro; YOUNG, Michal. Teste e análise de software: processo, princípios e técnicas. Tradução de Bernardo Copstein, Flávio Moreira de Oliveira. Porto Alegre: Bookman, 2008. ISBN 9788577803743.

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de software: uma abordagem profissional. 8. ed. Porto Alegre: McGraw Hill, 2016. 968 p. ISBN 9788580555332.

SOMMERVILLE, Ian. Engenharia de software. 8. ed. São Paulo: Pearson, 2007. 544 p. ISBN 9788543007915.

SCHACH, Stephen R. Engenharia de software: os paradigmas clássico e orientado a objetos. 7. ed. São Paulo: McGraw-Hill, 2009. 618 p. ISBN 9788577260454.

HUMBLE, Jez; FARLEY, David. Entrega contínua: como entregar software. Porto Alegre: Bookman, 2013. 496 p. ISBN 9788582601037.

BROOKSHEAR, J. Glenn. Ciência da computação: uma visão abrangente. 11. ed. Porto Alegre: Bookman, 2013. 576 p. ISBN 9788582601471.





OBRIGADO!

Professor: Leonardo



www.leonardotirado.com